GETTING STARTED 开始
Overview 概述
Quick start 快速入门
KEY CONCEPTS 关键概念
Architecture 建筑
Layouts 布局
Templates 模板
Sections 部分
Section groups 部分组
Blocks 块
Snippets 片段
Settings 设置
Overview 概述
Input settings 输入设置
Sidebar settings 边栏设置
Dynamic sources 动态源
Fonts 字体
Deprecated settings 已弃用的设置

Settings 设置

To make it easier for merchants to customize your theme, you can use JSON to create settings that merchants can access through the theme editor.

为了让商家更轻松地自定义您的模板，您可以使用 JSON 创建商家可以通过模板编辑器访问的设置。

You can provide settings at the theme, section, or block level. Settings can be fixed (such as informational elements) or interactive (such as a drop-down menu). Setting values can be static, or use dynamic sources to render contextually appropriate values.

您可以在主题、部分或块级别提供设置。设置可以是固定的（例如信息元素）或交互式的（例如下拉菜单）。设置值可以是静态的，也可以使用动态源来呈现上下文适当的值。

Exposing settings makes your theme more customizable so it can better express a merchant's brand. It also can make your theme more flexible so that you can address various use cases for merchants.

公开设置使您的主题更具可定制性，从而更好地表达商家的品牌。它还可以使您的主题更加灵活，以便您可以解决商家的各种用例。

Subtypes 亚
Location 位置
Schema 图式
Usage 用法
Conditional settings 条件设置
Platform-controlled settings 平台控制的设置
Translate settings 翻译设置

Subtypes 亚

There are two categories of settings:

设置分为两类：

Category 类别	Description 描述
Input settings 输入设置	Settings that can hold a value, and are configurable by app users. 可以保存值且可由应用用户配置的设置。
Sidebar settings 边栏设置	Settings that can't hold a value, and aren't configurable by app users. They're informational elements that can be used to provide detail and clarity for your input settings. 无法保存值且应用用户无法配置的设置。它们是信息元素，可用于为您的输入设置提供详细信息和清晰度。

Location 位置

You can create settings in the following places:

您可以在以下位置创建设置：

- `config` > `settings_schema.json`

  `配置` > `settings_schema.json`
- Section files in the `sections` folder, using the the section's `{% schema %}` tag

  `sections` 文件夹中的 section 文件，使用该部分的 `{% schema %}` 标记

```
└── theme
  ├── config
  |   ├── settings_schema.json
  |   ...
  ├── sections
  |   ├── main_product.liquid
  |   ├── another_section_file.liquid
  |   ...
  ...
```

settings_schema.json

The settings_schema.json file controls the content of the **Theme settings** area of the theme editor. Settings in this file translate to global theme settings, which can be accessed through the Liquid settings object.

settings_schema.json 文件控制主题编辑器的**主题设置**区域的内容。此文件中的设置转换为全局主题设置，可以通过 Liquid 设置对象访问。

Section schema 部分架构

The section `{% schema %}` tag is where you can create section settings and block settings. Those settings can be accessed through the `settings` attribute of the `section` object and `block` object, respectively.

部分 `{% schema %}` 标签是您可以创建部分设置和阻止设置的地方。这些设置可以分别通过 `section` 对象和 `block` 对象的 `settings` 属性访问。

Schema 图式

Settings are defined as a JSON `settings` attribute that's parented to the object that the settings apply to. This attribute accepts an array of settings. Input settings and sidebar settings both use standard schema attributes. You can find detailed descriptions of these attributes in their respective sections:

设置定义为一个 JSON `设置` 属性，该属性是应用设置的对象的父级。此属性接受一系列设置。输入设置和侧边栏设置都使用标准架构属性。您可以在各自的部分中找到这些属性的详细说明：

- Input Settings 输入设置
- Sidebar settings 边栏设置

Most setting types may be conditionally set using the `visible_if` attribute.

大多数设置类型都可以使用 `visible_if` 属性有条件地设置 。

Basic setting example 基本设置示例

```
{
  ...
  "settings": [
    {
      "type": "header",
      "content": "My settings"
    },
    {
      "type": "text",
      "id": "my_id",
      "label": "My setting label",
      "default": "Enter text here"
    },
    {
      "type": "select",
      "id": "layout_style",
      "label": "type",
      "options": [
        {
          "value": "flex",
          "label": "Stack"
        },
        {
          "value": "grid",
          "label": "Grid"
        }
      ],
```

Usage 用法

When working with settings, you should familiarize yourself with the following:

使用设置时，应熟悉以下内容：

- Accessing setting values

  访问设置值
- Checking the setting value format

  检查设置值格式
- Using dynamic sources for settings

  使用动态源进行设置

Access settings 访问设置

Depending on where they were created, you can access settings through the following Liquid objects:

根据它们的创建位置，您可以通过以下 Liquid 对象访问设置：

- The global `settings` object

  全局设置对象
- The `section` object

  `section` 对象
- The `block` object

  块对象

Note 注意

Settings from the `settings` object can be accessed in Liquid theme assets.

可以在 Liquid 主题资源中访问 `设置` 对象中的设置。

To access a specific setting, append the `id` attribute of the associated setting to the object that you want to access.

要访问特定设置，请将关联设置的 `id` 属性附加到要访问的对象。

For example, if you had the following setting implemented in each Liquid object:

例如，如果您在每个 Liquid 对象中实现了以下设置：

```
{
  "type": "text",
  "id": "message",
  "label": "Message",
  "default": "Hello!"
}
```

Then the following Liquid would generate the following output:

然后，以下 Liquid 将生成以下输出：

Input 输入

```
// Settings
Message: {{ settings.message }}

// Section
Message: {{ section.settings.message }}

// Block
Message: {{ block.settings.message }}
```

Output 输出

```
// Settings
Message: Hello!

// Section
Message: Hello!

// Block
Message: Hello!
```

Check the format of the setting value
检查设置值的格式

When referencing settings, you should always check that the value is in the format that you expect. Any setting without an automatic default value could end up with no value, which translates to an empty string.

引用设置时，应始终检查值是否采用预期格式。任何没有自动默认值的设置最终都可能没有值，这会转换为空字符串 。

For example, if you have a setting with an `id` of `message`, then the following Liquid would generate the following output depending on the value:

例如，如果您的设置 `ID 为` `message`，则以下 Liquid 将根据该值生成以下输出：

Input 输入

```
// No value
Setting: {{ settings.message }}

// With value
Setting: {{ settings.message }}
```

Output 输出

```
// No value
Setting:

// With value
Setting: Message value
```

You can check whether a value is an empty string with the `blank` operator. For example:

您可以使用 `空白` 运算符检查值是否为空字符串。 例如：

```
{% unless settings.message == blank %}
  {{ settings.message }}
{% endunless %}
```

Resource-based settings 基于资源的设置

To avoid an empty string, check that the value is in the format that you expect. It's possible that no resource was selected, selected resource no longer exists, or the selected resource has been hidden.

若要避免出现空字符串，请检查值是否采用预期的格式。可能未选择任何资源、所选资源不再存在或所选资源已隐藏。

For example, if you have the following `page` type setting:

例如，如果您有以下 `页面` 类型设置：

```
{
  "type": "page",
  "id": "page",
  "label": "Page"
}
```

Then you can check for emptiness like the following:

然后，您可以检查空虚情况。如下所示：

```
{% if settings.page != blank %}
  {{ settings.page.title }}
  {{ settings.page.content }}
{% else %}
  No page, or invalid page, selected.
{% endif %}
```

Tip 提示

Resource-based settings didn't always return the resource object. To learn more, refer to Legacy resource-based settings.

基于资源的设置并不总是返回资源对象。若要了解详细信息，请参阅旧版基于资源的设置 。

Legacy resource-based settings
旧版基于资源的设置

In the past, resource-based settings returned the handle of the associated resource, and you had to access the actual object through Liquid using that handle.

过去，基于资源的设置会返回关联资源的句柄，您必须使用该句柄通过 Liquid 访问实际对象。

For example, if you had the following product setting, then you would need to access the product object like the following:

例如，如果您有以下产品设置，则需要访问产品对象，如下所示：

Setting 设置

```
{
  "type": "product",
  "id": "product",
  "label": "Product"
}
```

Access setting 访问设置

```
{% unless settings.product == blank %}
  {% assign product = all_products[settings.product] %}

  {% if product %}
    {{ product.title }} - {{ product.price }}
  {% else %}
    No product, or invalid product, selected.
  {% endif %}
{% endunless %}
```

Dynamic sources 动态源

Settings for sections and blocks included in a JSON template have the option for merchants to connect one or more dynamic sources to the setting, depending on the setting type.

对于 JSON 模板中包含的版块和块的设置，商家可以选择将一个或多个动态源连接到该设置，具体取决于设置类型。

Learn more about dynamic sources.

了解有关动态源的更多信息 。

Conditional settings 条件设置

Settings can be displayed conditionally by passing a boolean expression to the `visible_if` attribute:

可以通过将布尔表达式传递给 `visible_if` 属性来有条件地显示设置：

```
"visible_if": "{{ block.settings.layout_style == 'flex' }}"
```

Not all settings can be conditionally set. The following settings support conditional settings:

并非所有设置都可以有条件地设置。以下设置支持条件设置：

- All basic input settings

  所有基本输入设置
- All sidebar settings

  所有边栏设置
- These specialized input settings:

  这些专用输入设置 ：
  - color 颜色
  - color_background
  - color_scheme
  - font_picker
  - html [HTML 全文]
  - image_picker
  - inline_richtext
  - link_list
  - liquid 液
  - richtext 富文本
  - text_alignment
  - url 网址
  - video 视频
  - video_url

Note 注意

Conditional settings cannot access runtime context or resolved data source values. While you can check if a setting with a data source *has a value*, you cannot create conditions based on what that data source *resolves to*.

条件设置无法访问运行时上下文或已解析的数据源值。虽然可以检查具有数据源的设置是否*具有值*，但不能根据该数据源*解析为什么*创建条件。

Platform-controlled settings
平台控制的设置

In the theme editor, Shopify exposes a custom CSS setting at the theme and section level. You can't add or hide this setting in your settings schema.

在模板编辑器中，Shopify 会在模板和版块级别公开自定义 CSS 设置 。您无法在设置架构中添加或隐藏此设置。

Any custom CSS that merchants add using this setting is stored in a `custom_css` attribute, either in a JSON template's section attribute, or in the settings_data.json `platform_customizations` object.

商家使用此设置添加的任何自定义 CSS 都存储在 `custom_css` 属性中，可以是 JSON 模板的 section 属性 ，也可以存储在 settings_data.json `platform_customizations` 对象中。

This setting is intended to enable users to customize the look and feel of their storefront without editing theme code. As a theme developer, you shouldn't add this setting, or edit the value of this setting after it's set. Instead, you should use dedicated CSS assets and `stylesheet` Liquid tags, and introduce customization options for CSS in these areas using theme settings.

此设置旨在使用户能够自定义其店面的外观，而无需编辑主题代码。作为主题开发人员，您不应添加此设置，也不应在设置后编辑此设置的值。相反，您应该使用专用的 CSS 资源和`样式表` Liquid 标签 ，并使用主题设置在这些区域引入 CSS 的自定义选项。

Translate settings 翻译设置

You can translate various attributes of the settings schema depending on the online store's active language. These translations are stored in schema locale files.

您可以根据在线商店的活动语言翻译设置架构的各种属性 。这些翻译存储在架构区域设置文件中。

Was this page helpful? 此页面对您有帮助吗？ Yes 是的 No 不